

# Integration Table Template

**Day 1 · Activity 3 handout.** Begin TCD §2 by listing every system your feature depends on or extends. The **Failure handling** column forces a choice teams usually defer — make it explicit now.

## The five columns

For each integration, capture:

| SYSTEM / SERVICE | OWNER TEAM | SYNCHRONOUS OR ASYNC? | READ / WRITE / BOTH | FAILURE HANDLING |
|------------------|------------|-----------------------|---------------------|------------------|
|                  |            |                       |                     |                  |
|                  |            |                       |                     |                  |
|                  |            |                       |                     |                  |
|                  |            |                       |                     |                  |

## Reference row shapes (FieldPulse examples)

| SYSTEM / SERVICE     | OWNER TEAM | SYNCHRONOUS OR ASYNC? | READ / WRITE / BOTH | FAILURE HANDLING         |
|----------------------|------------|-----------------------|---------------------|--------------------------|
| Auth (SSO)           | Identity   | Sync                  | Read                | Fail open / fail closed? |
| Audit event store    | Platform   | Async                 | Write               | Drop or queue?           |
| Tickets API          | Dispatch   | Sync                  | Both                | Retry policy?            |
| Notification service | Comms      | Async                 | Write               | Drop / queue / DLQ?      |

## Your protocol

1. **List integrations** (15 min). Draw from PRD §10 plus anything you forgot.
2. **Characterize each** (15 min). Sync/async, read/write, failure mode.
3. **Identify the "two-way contracts"** (10 min). Which integrations require coordination with another team? Mark these — they go on the dependency owner list.

## Notes

- **"Failure handling: TBD"** is fine if it's an open question for the architect. **"Failure handling: hopefully it works"** is not.
- Watch **read-then-write** dependencies — they're the most common source of consistency bugs. Flag them.

Two-way contracts (require another team's coordination)

- ---
- ---
- ---